

Tutoriel : Prompt Engineering & Function Calling

Part 3 – Des Transformers aux Large Language Models

Table des matieres

1. [Introduction](#)
 2. [Les fondamentaux du Prompt Engineering](#)
 3. [Chain-of-Thought \(CoT\) – Raisonnement etape par etape](#)
 4. [Techniques avancees de prompting](#)
 5. [Extraction structuree \(mode JSON\)](#)
 6. [Function Calling – Appel de fonctions](#)
 7. [Evaluation des strategies de prompting](#)
 8. [Bonnes pratiques et resume](#)
 9. [References](#)
-

1. Introduction

Le **prompt engineering** est l'art de formuler des instructions pour obtenir les meilleures reponses possibles d'un LLM. Un meme modele peut donner des resultats mediocres ou excellents selon la maniere dont on lui pose la question.

Ce tutoriel couvre les techniques essentielles, de la simple instruction au function calling, en passant par le raisonnement en chaine.

Outils utilises : Python, Hugging Face Transformers, Matplotlib, Pandas

2. Les fondamentaux du Prompt Engineering

2.1 System Prompts – Definir le role du modele

Le **system prompt** est le levier le plus puissant pour controler le comportement d'un LLM. Il definit le persona, le style et les contraintes de la reponse.

Principe : On donne au modele un role precis avant de poser la question.

```
system_prompts = {  
    "Expert":      "Tu es un professeur expert en ML. Explique avec des  
analogies.",  
    "Enfant":      "Explique comme si tu parlais a un enfant de 5 ans.",  
    "Ingenieur":   "Sois extremement concis – 1 a 2 phrases maximum.",
```

```
"Socratique": "Ne donne jamais la reponse. Pose des questions pour guider.",
}
```

Impact : Pour une meme question ("Qu'est-ce que la descente de gradient ?"), chaque persona produit une reponse radicalement differente en ton, longueur et niveau de detail.

A retenir : Le system prompt est la premiere chose a configurer. C'est le parametre qui a le plus d'influence sur la qualite de la reponse.

2.2 Zero-Shot vs. Few-Shot Prompting

Zero-Shot

On pose la question directement, sans exemple. Le modele utilise uniquement ses connaissances pre-entrainees.

Classifie ce texte : "Le nouvel iPhone 16 a une puce A18 plus rapide."
Categorie :

Few-Shot (apprentissage en contexte)

On fournit quelques exemples (2 a 5) avant de poser la question. Le modele apprend le format et la logique attendus **dans le contexte meme du prompt**.

Texte : "Tesla a annonce des revenus records." → Finance

Texte : "Les chercheurs du MIT ont developpe un algorithme." → Science

Texte : "Le nouvel iPhone 16 a une puce A18 plus rapide." → ???

Resultat cle : 2 a 5 exemples suffisent generalement pour ameliorer significativement la precision, surtout pour la classification et l'extraction.

Reference : Brown et al. 2020 – travail fondateur sur l'apprentissage en contexte (in-context learning).

2.3 Templates de prompts reutilisables

Plutot que de reecire les prompts a chaque fois, on definit des **templates parametres** :

Template	Usage	Exemple de parametres
Classification	Categoriser un texte	{categories}, {text}
Extraction	Extraire des entites	{fields}, {text}
Resume	Condenser un texte	{length}, {text}
Question-Reponse	Repondre selon un contexte	{context}, {question}

Bonnes pratiques pour les templates :

- Etre explicite sur le format de sortie attendu
- Utiliser des separateurs clairs entre les sections
- Specifier les contraintes (longueur, format, options)

3. Chain-of-Thought (CoT) – Raisonnement etape par etape

3.1 Le probleme

Les LLMs echouent souvent sur les problemes de raisonnement quand on leur demande une reponse directe. Ils "sautent" les etapes intermediaires.

3.2 Zero-Shot CoT

La technique la plus simple : ajouter **"Reflechissons etape par etape"** a la fin du prompt.

Question : Une boulangerie fait 120 croissants. Elle en vend 75% avant midi et 60% du reste l'apres-midi. Combien en reste-t-il ?

Reflechissons etape par etape.

Cout : Zero – une seule phrase ajoutee.

Gain : Amelioration significative sur les problemes mathematiques et logiques.

Reference : Kojima et al. 2022 ; Wei et al. 2022

3.3 Few-Shot CoT

On combine le few-shot avec le CoT en fournissant des **exemples resolus avec le raisonnement complet** :

Q : Un magasin a 50 pommes. Il en vend 30% le matin et 40% du reste l'apres-midi.

A : Resolvons etape par etape.

1. Ventes du matin : $50 \times 0.30 = 15$
 2. Apres le matin : $50 - 15 = 35$
 3. Ventes de l'apres-midi : $35 \times 0.40 = 14$
 4. Reste : $35 - 14 = 21$
- La reponse est 21.

Q : [nouveau probleme]

A : Resolvons etape par etape.

C'est le **gold standard** pour les taches de raisonnement complexe.

3.4 Self-Consistency – Vote majoritaire

Idee : Au lieu de faire confiance a une seule chaine de raisonnement, on en genere **plusieurs** (avec temperature > 0) et on prend la **reponse majoritaire**.

Algorithme :

1. Generer N reponses avec temperature elevee (ex. 0.8)
2. Extraire la reponse finale de chaque chaine
3. Voter : la reponse la plus frequente gagne
4. Confiance = nombre de votes / N

Exemple :

- Chaine 1 → 11
- Chaine 2 → 11
- Chaine 3 → 12
- Chaine 4 → 11
- Chaine 5 → 11
- **Vote majoritaire : 11** (confiance = 4/5 = 80%)

Reference : Wang et al. 2022

4. Techniques avancees de prompting

4.1 Tree-of-Thoughts (ToT)

Le CoT suit un seul chemin de raisonnement. Le **Tree-of-Thoughts** explore **plusieurs branches** en parallele, puis evalue chacune pour choisir la meilleure.

Etape 1 – Brainstormer : "Genere 3 approches differentes pour resoudre ce probleme."

Etape 2 – Evaluer : "Evalue chaque approche. Laquelle est la plus fiable ?"

Etape 3 – Selectionner : Le modele choisit la meilleure branche.

Ideal pour : les problemes creatifs ou les raisonnements complexes ou un seul chemin peut etre trompeur.

Reference : Yao et al. 2023

4.2 Prompt Chaining – Decomposition en sous-taches

Plutot qu'un seul prompt monolithique, on **decompose une tache complexe en etapes sequentielles**, ou la sortie de l'etape N devient l'entree de l'etape N+1.

Etape 1 : Extraire les informations clés d'un article

↓ (resultat)

Etape 2 : Generer des questions critiques basees sur l'extraction

↓ (resultat)

Etape 3 : Rediger un resume synthetique (tweet)

Avantages :

- Chaque etape est simple et verifiable
- On peut ajuster chaque maillon independamment
- C'est la **base des workflows agentiques**

5. Extraction structuree (mode JSON)

5.1 Principe

Pour integrer les reponses d'un LLM dans un pipeline automatise, on a besoin de **sorties structurees** (JSON, pas du texte libre).

Technique : Instruire le modele dans le prompt de repondre uniquement en JSON valide, avec des clés specifiees.

```
prompt = """Extrais les informations de cet avis client.  
Retourne un objet JSON avec ces clés :  
– product: string  
– price: number  
– rating: number  
– pros: liste de strings  
– cons: liste de strings
```

```
Avis : "J'ai achete les AirPods Pro 2 pour 249€. La reduction de bruit  
est incroyable mais l'autonomie pourrait etre meilleure. 4/5 etoiles."  
"""
```

5.2 Extraction par lots (Batch)

On peut appliquer le meme template d'extraction a **plusieurs entrees** en boucle pour construire un dataset structure (DataFrame).

5.3 Parsing robuste avec fallbacks

En production, le JSON genere n'est pas toujours valide. On utilise une strategie de **retry avec fallback** :

1. Essayer `json.loads(reponse)`
2. Si echec → chercher un **bloc JSON** avec regex dans la reponse

3. Si echec → reformuler le prompt ("Tu DOIS répondre en JSON valide")
4. Repeter jusqu'à max_retries

6. Function Calling – Appel de fonctions

6.1 Concept

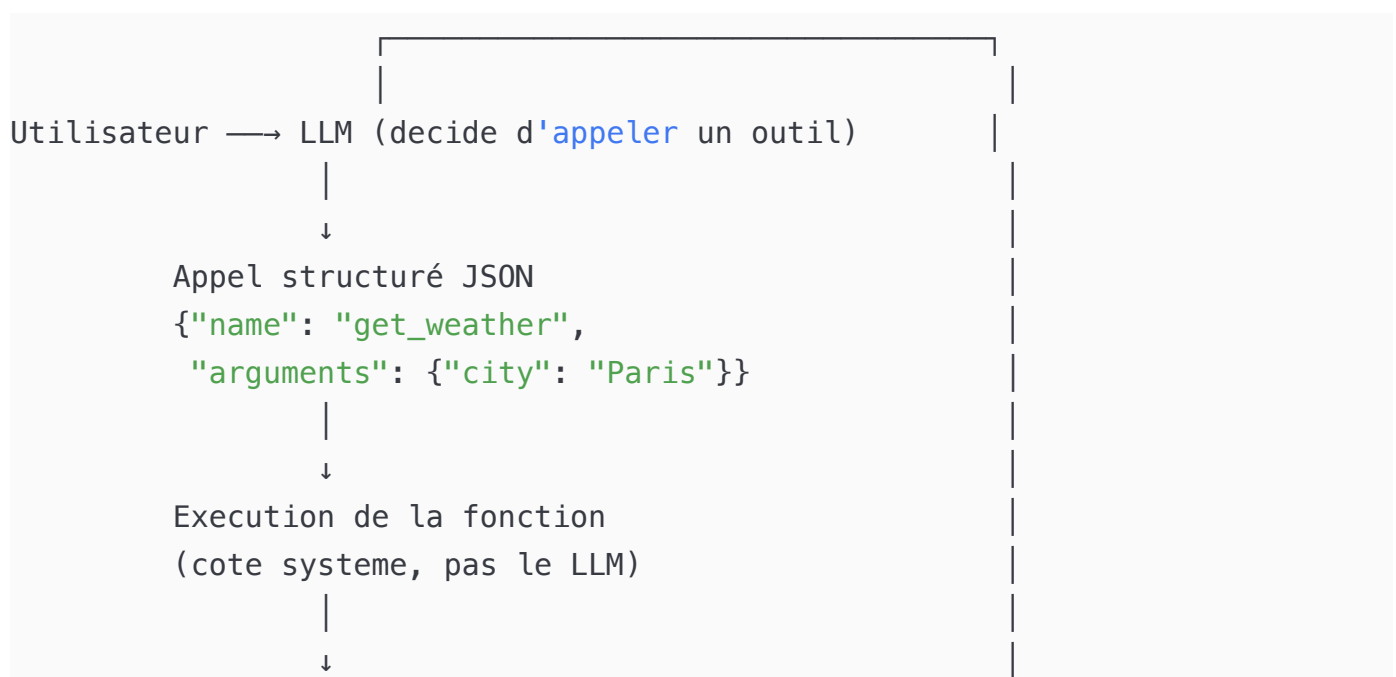
Le **function calling** permet a un LLM de **decider quand et comment appeler des outils externes**.
Le modele ne les execute pas – il genere un appel structure (JSON) que le systeme execute.

6.2 Definition des outils (JSON Schema)

Chaque outil est decrit par un schema JSON :

```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Obtenir la meteo d'une ville",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {"type": "string", "description": "Nom de la ville"},
        "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
      },
      "required": ["city"]
    }
  }
}
```

6.3 Boucle complete de function calling



```
Resultat → renvoie au LLM _____  
|  
↓  
Reponse finale a l'utilisateur :  
"Il fait 18°C a Paris, partiellement nuageux."
```

6.4 Appels paralleles

Un meme message peut declencher **plusieurs appels simultanes** :

```
"Compare la meteo a Tokyo et Londres"  
→ get_weather(city="Tokyo")  
→ get_weather(city="London")
```

6.5 Function calling par prompt (sans support natif)

Si le modele ne supporte pas le function calling nativement, on peut le **simuler via le prompt** en demandant au modele de repondre en JSON avec le nom de la fonction et ses arguments.

```
Si tu as besoin d'un outil, reponds UNIQUEMENT avec :  
{"tool": "nom_outil", "arguments": {"arg1": "valeur1"}}  
Sinon, reponds normalement.
```

7. Evaluation des strategies de prompting

7.1 Benchmark sur des problemes mathematiques

On compare systematiquement les strategies sur des problemes a reponse connue :

Strategie	Complexite	Performance typique
Standard (reponse directe)	Minimale	Faible sur le raisonnement
Zero-shot CoT	Faible (+1 phrase)	Amelioration significative
Few-shot CoT	Moyenne (exemples)	Meilleure precision
Self-Consistency	Elevee (N appels)	La plus fiable

7.2 Benchmark sur la classification de sentiments

On teste differentes formulations de prompt sur la meme tache :

Variante	Description
Minimal	Sentiment de : "{texte}" →
Instruit	Instructions explicites + categories

Variante	Description
Few-shot	3 exemples etiquetes
CoT	"Identifie les mots emotionnels, puis classifie"

Lecon cle : Il faut **toujours mesurer** – l'intuition seule est peu fiable. Un prompt qui semble meilleur peut etre moins performant en pratique.

7.3 Impact de la taille du modele

Les petits modeles (~360M parametres) sont **plus sensibles a la qualite du prompt** que les grands modeles. Les techniques comme le few-shot et le CoT sont encore plus importantes quand le modele est petit.

8. Bonnes pratiques et resume

Les 5 principes fondamentaux

#	Principe	Explication
1	Etre specifique	Eviter les instructions vagues. Dire exactement ce qu'on attend.
2	Montrer, ne pas dire	Des exemples (few-shot) valent mieux que des instructions longues.
3	Contraindre la sortie	Specifier le format (JSON, liste, 1 phrase), la longueur, les options.
4	Utiliser des separateurs	Delimiter clairement les sections (contexte, question, instructions).
5	Iterer et evaluer	Toujours benchmarker sur des reponses connues.

Tableau recapitulatif des techniques

Technique	Quand l'utiliser	Cout
System Prompt	Toujours – c'est la base	Aucun
Zero-shot CoT	Raisonnement, maths, logique	+1 phrase
Few-shot	Classification, extraction, format precis	+quelques exemples
Few-shot CoT	Raisonnement complexe	+exemples resolus
Self-Consistency	Quand la fiabilite est critique	N x appels
Tree-of-Thoughts	Problemes creatifs ou ambigus	2 appels

Technique	Quand l'utiliser	Cout
Prompt Chaining	Taches complexes multi-etapes	Pipeline
Function Calling	Integration avec des outils/API	Schema JSON
Mode JSON	Sorties structurees pour pipelines	Instruction format

9. References

Article	Auteurs	Contribution
Language Models are Few-Shot Learners	Brown et al., 2020	Apprentissage en contexte (few-shot)
Chain-of-Thought Prompting Elicits Reasoning	Wei et al., 2022	Raisonnement etape par etape
Large Language Models are Zero-Shot Reasoners	Kojima et al., 2022	Zero-shot CoT ("Let's think step by step")
Self-Consistency Improves CoT Reasoning	Wang et al., 2022	Vote majoritaire sur plusieurs chaines
Tree of Thoughts: Deliberate Problem Solving	Yao et al., 2023	Exploration multi-branches

Note : Ce tutoriel accompagne le Lab 5 du cours "Des Transformers aux LLMs". Pour la pratique, se referer au notebook `lab5-prompt-engineering.ipynb`.